# Zero Trust Security with OCI

**OUT1015 Hands-On Labs: Activity Guide**

# Table of Contents

**Zero Trust Security with OCI** ..... **1**
Introduction ..... 3
About This Workshop ..... 4
Architecture ..... 4
The Summit Retail Incidents ..... 5
Objectives ..... 5
Get Started ..... 6
Prerequisites ..... 6
Sign In to the Lab Tenancy ..... 6
Sign In and Check Your Region ..... 6
Find Your Participant Number ..... 6
How to Read This Guide ..... 7
Practice 0: Prepare Your Summit Retail Environment ..... 8
Introduction ..... 8
Objectives ..... 8
Practice 0-1: Open Cloud Shell and Download the Package ..... 8
Practice 0-2: Prepare Your Settings ..... 9
Practice 0-3: Review the Plan ..... 10
Practice 0-4: Deploy the Environment ..... 10
Practice 0-5: Check Your Environment ..... 11
Learn More ..... 11
Practice 1: Get the Password Out of the Code ..... 12
Introduction ..... 12
Objectives ..... 12
Prerequisites ..... 12
Practice 1-1: Create Your Master Encryption Key ..... 12
Practice 1-2: Store the Password as a Secret ..... 14
Practice 1-3: See How the Instance Is Authorized ..... 16
Practice 1-4: Connect to the Tools VM ..... 17
Practice 1-5: Retrieve the Password at Runtime ..... 18
Practice 1-6 (Optional): Retrieve the Secret with the OCI CLI ..... 19
Checkpoint: What You Proved ..... 19
Learn More ..... 20
Practice 2: Keep the Private Application Private ..... 21
Introduction ..... 21
Objectives ..... 21
Prerequisites ..... 21
Practice 2-1: Set Up Your Values in Cloud Shell ..... 21
Practice 2-2: Use the Customer Path ..... 22
Practice 2-3: Find the Rules That Apply to the Private Subnet ..... 22
Practice 2-4: Check the Application's Network Security Group ..... 23

Copyright © 2026, Oracle and/or its affiliates.

Practice 2-5: Maintain the Application Through Bastion....24
Practice 2-6: Reproduce the Incident....27
Practice 2-7: Remove the Broad Rule....28
Practice 2-8: End the Administrator Session....29
Checkpoint: What You Proved....29
Learn More....30
Practice 3: Inspect the Requests, Not Only the Port....31
Introduction....31
Objectives....31
Prerequisites....31
Practice 3-1: Meet the Working Shop....31
Practice 3-2: Create a WAF Policy on the Load Balancer....32
Practice 3-3: Add an XSS Rule in Check Mode....33
Practice 3-4: Turn On the WAF Log....35
Practice 3-5: Compare Two Requests in Check Mode....35
Practice 3-6: Block the Pattern....36
Practice 3-7: Limit Repeated Requests....38
Practice 3-8: Read the Evidence....40
Checkpoint: What You Proved....42
Learn More....43
Need Help?....44
Practice 0....44
Practice 1....44
Practice 2....44
Practice 3....45
Clean Up....46
Check for Leftover Lab Changes....46
Remove Your WAF Policy and Log....46
Delete Your Secret and Key....46
Remove the Practice 0 Environment....47
Acknowledgements....48

Copyright © 2026, Oracle and/or its affiliates.

# Introduction

## About This Workshop

Summit Retail is moving its online store to Oracle Cloud Infrastructure (OCI). Asha leads cloud security for the move. Over one day she handles a series of security incidents, and each one leads to a Zero Trust decision: grant access on purpose, verify every request, and assume that one control can fail.

You watch some incidents as instructor demonstrations, and you fix three of them yourself. Each practice starts with what Asha sees, shows you the part of the architecture involved, and then gives you short tasks. After each task you compare your result with the expected result.

**Estimated workshop time:** about 1 hour 40 minutes of hands-on work, spread across the day.

## Architecture

Practice 0 builds everything in the diagram except the vault, which the lab team prepared, and the WAF policy, which you add in Practice 3. The badges show where each practice works.

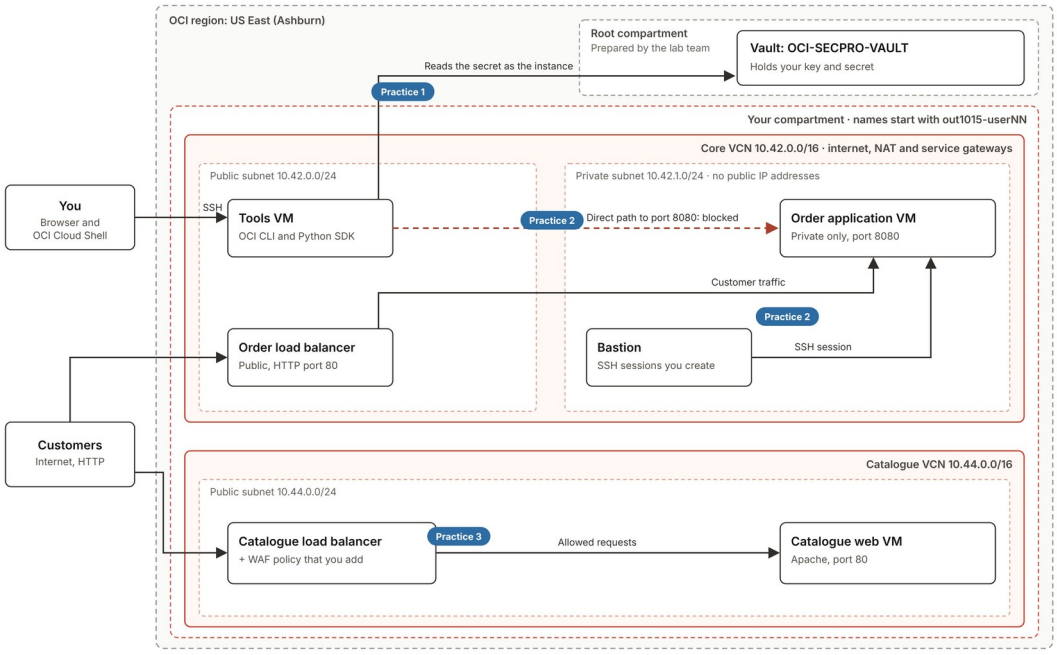



*Figure 1: The Summit Retail lab environment in your compartment*

The core VCN holds the order system. Customers reach the order application only through the public load balancer. The application server has no public IP address, and you reach it for administration only through OCI Bastion. The Tools VM plays the role of an administrator's or

Copyright © 2026, Oracle and/or its affiliates.

another team's machine inside the same VCN. A second VCN holds the product catalogue, which you protect with WAF.

## The Summit Retail Incidents

| Incident | What happens | How you take part |
|---|---|---|
| 1. Identity | A user with read access stops a production instance. Asha finds the grant behind it, adds an explicit deny, and moves sign-in to single sign-on. | Instructor demonstration |
| 2. Secrets | The order application's database password sits in its code. Asha moves it to OCI Vault and lets the instance read it with its own identity. | **Practice 1** (hands-on) |
| 3. Network | The private order application can be reached on a path nobody intended. Asha uses OCI Bastion, finds the broad rule, and removes it. | **Practice 2** (hands-on) |
| Business path | Zero Trust Packet Routing (ZPR) lets only the intended business path reach the database. | Instructor demonstration |
| 4. Web requests | Attack patterns and floods of requests arrive on the shop's open web port. Asha adds OCI Web Application Firewall (WAF). | **Practice 3** (hands-on) |
| 5. Data | A test database holds real customer data. Asha finds it and masks it with Oracle Data Safe. | Instructor demonstration |
| 6. Posture | A storage bucket becomes public. Cloud Guard detects it, and Security Zones prevent it from happening again. | Instructor explanation |

## Objectives

In the hands-on practices, you:

- Deploy the Summit Retail environment from OCI Cloud Shell with a prepared Terraform package.
- Move a password into OCI Vault and retrieve it at runtime with an instance principal instead of a stored credential.
- Reach a private server through OCI Bastion, then find and remove a network rule that exposes it.
- Detect, block, and rate-limit web requests with OCI Web Application Firewall, and confirm the results in the logs.

Copyright © 2026, Oracle and/or its affiliates.

# Get Started

## Prerequisites

- The lab account details from your instructor: the sign-in URL or tenancy name, your username and password, and your assigned compartment.
- A laptop with a current web browser. You need no other software, because every command runs in OCI Cloud Shell.

## Sign In to the Lab Tenancy

Your instructor gives you a lab account in a shared tenancy. Every participant works in their own compartment, so your resources never mix with anyone else's.

## Sign In and Check Your Region

1. Open the sign-in URL from your lab details, or go to `https://cloud.oracle.com` and enter the tenancy name.

2. Sign in with the username and password from your lab details. If you are asked to change the password, choose a new one and keep it for the rest of the day.

3. In the Console header, open the region selector and select **US East (Ashburn)**. Every practice uses this region.

4. From the navigation menu, select **Identity & Security**, and then click **Compartments**. Find your assigned compartment in the list, for example `98507690-C07`.

   > **Expected result**
   >
   > You are signed in, the region is US East (Ashburn), and you can see your assigned compartment.

## Find Your Participant Number

1. Look at your username. It ends with your participant number, for example `98507690-lab.user07`.

2. Write down the last part, `userNN`, for example `user07`. Everywhere this guide shows `userNN` or `NN`, use your own number.

Copyright © 2026, Oracle and/or its affiliates.

> **Note**
>
> Your compartment uses the same number: user07 works in compartment 98507690-C07. The Practice 0 package reads both from your username, and asks you to confirm them.

## How to Read This Guide

| You see | It means |
|---|---|
| A light box with a red left edge | Text that you enter or select in the OCI Console. |
| A dark box that starts with $ | A command to run. The task tells you where: in Cloud Shell or on a VM. Do not type the $. |
| A gray box | Output that you should see. Details such as IDs, dates, and IP addresses differ for you. |
| **Expected result** | What to check before you move on. |
| `<value>` | A placeholder. Replace it with your own value, without the angle brackets. |

> **Note**
>
> Console labels sometimes move between releases. If a button is not where the guide says, look for the same action in the **Actions** menu or under the three-dot icon.

Copyright © 2026, Oracle and/or its affiliates.

# Practice 0: Prepare Your Summit Retail Environment

## Introduction

Before Asha can fix anything, Summit Retail's systems must be running. In this practice you build them in your compartment with one prepared Terraform package: two VCNs, three virtual machines, two load balancers, and a bastion. The labs that follow change security controls only, so you build this once.

**Estimated time:** 20 minutes, most of it waiting while the servers start.

## Objectives

- Download the setup package into OCI Cloud Shell.
- Confirm your participant number and compartment, and review the plan before anything is created.
- Deploy the environment and check that both web applications respond.

## Practice 0-1: Open Cloud Shell and Download the Package

1. In the Console header, click the **Developer tools** icon and select **Cloud Shell**. Wait until the prompt appears at the bottom of the window.

2. Download the package and list its contents:

```
$ REPO=https://github.com/SaurabhOCI/aiworld-2026-zero-trust-labs
$ ZIP=OUT1015_Participant_Foundation_v6.zip
$ wget -q -O ~/$ZIP $REPO/raw/main/lab0/$ZIP
$ unzip -l ~/$ZIP | tail -1
```

```
    46119                   16 files
```

3. If the last line is not `16 files`, download the same package from Oracle Object Storage instead, and list it again:

```
$ OS=https://objectstorage.us-ashburn-1.oraclecloud.com
$ P=7-3XsOEt1Q48HA2iREfVy1ZpHN94urCBIa
$ P=${P}ySLK6lMScVxL36NTdpqJZ4hdCYCJc9
$ B=n/intoraclerohit/b/out1015-aiworld-2026-labs
$ wget -q -O ~/$ZIP $OS/p/$P/$B/o/lab0/$ZIP
$ unzip -l ~/$ZIP | tail -1
```

```
    46119                   16 files
```

Copyright © 2026, Oracle and/or its affiliates.

4. Extract the package into a new folder and move into it:

```
$ unzip -n ~/OUT1015_Participant_Foundation_v6.zip -d ~/out1015-lab
$ cd ~/out1015-lab
```

> **Note**
>
> Keep the `~/out1015-lab` folder for the whole workshop. It holds your SSH key and the record of what the package created, which it needs for clean-up.
>
> If both downloads fail, ask your instructor for the zip file, and upload it with the Cloud Shell menu (**Upload**).

## Practice 0-2: Prepare Your Settings

1. Run the preparation step:

```
$ python3 lab.py prepare
```

2. The package shows your username and proposes your participant number and compartment. Check them, and press **Enter** to accept each value. Type a different value only if the proposal is wrong.

```
Signed-in user: 98507690-lab.user07
Participant number (userNN) [user07]:
Your assigned compartment name [98507690-C07]:
```

3. When the package lists the availability domains, press **Enter** to accept availability domain 1.

> **Expected result**
>
> The last lines read:
>
> ```
> SSH private key stays in Cloud Shell: /home/<you>/out1015-
> lab/.local/workshop_ssh
> No infrastructure created. Next: python3 lab.py plan
> ```

> **Note**
>
> The package creates an SSH key pair for you. The private key never leaves Cloud Shell. You use it to connect to the Tools VM in Practice 1, and Bastion uses its public half in Practice 2.

Copyright © 2026, Oracle and/or its affiliates.

## Practice 0-3: Review the Plan

1. Ask Terraform what it would create. Nothing is created yet:

```
$ python3 lab.py plan
```

2. Scroll through the plan if you like. Every name starts with `out1015-userNN`. Then check the summary at the end:

```
Reviewed plan scope: {"create": 48, "update": 0, "delete": 0}
```

> **Note**
>
> Just above the summary, Terraform suggests running `terraform apply`. Ignore that line, and use the package's own command in Practice 0-4.
>
> The package refuses a plan that would change or delete anything, or touch another compartment. This protects your environment and everyone else's.

## Practice 0-4: Deploy the Environment

1. Apply the plan that you reviewed:

```
$ python3 lab.py apply
```

2. Wait for Terraform to finish. It takes about 5 minutes; the load balancers take the longest. The command then checks the servers once:

```
Apply complete! Resources: 48 added, 0 changed, 0 destroyed.
...
Resources created. The servers now finish starting; checking them:
```

> **Note**
>
> Right after `apply`, the check usually reports `not verified` or `Not ready yet`. That is expected: the servers are still installing their software. Continue with Practice 0-5.

> **Caution**
>
> If `apply` stops with a limit, quota, or capacity error, do not run it again in another folder. Keep this folder as it is, and tell your instructor.

Copyright © 2026, Oracle and/or its affiliates.

## Practice 0-5: Check Your Environment

1. The servers install their software after they start. Check the environment:

```
$ cd ~/out1015-lab
$ python3 lab.py status
```

2. If the last line reads `Not ready yet`, wait a minute and run the command again. The private VM can take up to 15 minutes after `apply` before its Bastion plugin runs. When everything is ready, the output looks like this:

```
Your Summit Retail environment (all values are saved in
.local/outputs.json):
  tools_public_ip  150.136.72.83
  app_private_ip   10.42.1.207
  order_url        http://129.80.254.190
  catalogue_url    http://157.151.217.8
Orders: HTTP 200; expected body matched: True
Catalogue: HTTP 200; expected body matched: True
Bastion plugin on the order application VM: RUNNING
Environment ready.
```

3. Copy the four values at the top into a text file. Your addresses differ from the example; the practices use them:

| Value | Used for |
|---|---|
| `tools_public_ip` | Connecting to the Tools VM with SSH (Practices 1 and 2) |
| `app_private_ip` | The order application's private address (Practice 2) |
| `order_url` | The public order site (Practice 2) |
| `catalogue_url` | The public product catalogue (Practice 3) |

4. Open the `catalogue_url` in a new browser tab. The Summit Retail catalogue shows three products.

> **Expected result**
>
> `status` reports **Environment ready.**, and the catalogue page opens in your browser.

## Learn More

- OCI Cloud Shell: https://docs.oracle.com/en-us/iaas/Content/API/Concepts/cloudshellintro.htm
- Terraform provider for OCI: https://docs.oracle.com/en-us/iaas/Content/dev/terraform/home.htm

Copyright © 2026, Oracle and/or its affiliates.

# Practice 1: Get the Password Out of the Code

## Introduction

Summit Retail's order application needs a database password, and today the password sits in the application's configuration file. During a support case, someone attached that file to a ticket, and the password went with it. Every copy of the file is one more place where the password can leak.

Asha's fix: store the password once, as a secret in OCI Vault, encrypted with a key that Summit Retail controls. The application keeps only the secret's Oracle Cloud ID (OCID), which is a reference, not the password. When the application runs on an OCI instance, it proves who it is as that instance, an instance principal, and fetches the password when it needs it. No password and no API key are stored on the server.

**Estimated time:** 25 minutes.

## Objectives

- Create a master encryption key and store a dummy password as a secret in the shared vault.
- Read the policy that lets your instance read secrets, and see how a dynamic group recognizes the instance.
- Run the order application on the Tools VM and watch it retrieve the password with its instance principal.

## Prerequisites

- Practice 0 is complete, and you have your `tools_public_ip`.
- The lab team prepared the shared vault `OCI-SECPRO-VAULT`, the dynamic group, and the IAM policy. You use them, but never change them.

## Practice 1-1: Create Your Master Encryption Key

1. From the navigation menu, select **Identity & Security** > **Key Management**, and then click **Vault**.

2. Next to **Applied filters**, make sure that the root compartment is selected. From the list of vaults, click **OCI-SECPRO-VAULT**. Check that the region is **US East (Ashburn)**.

Copyright © 2026, Oracle and/or its affiliates.

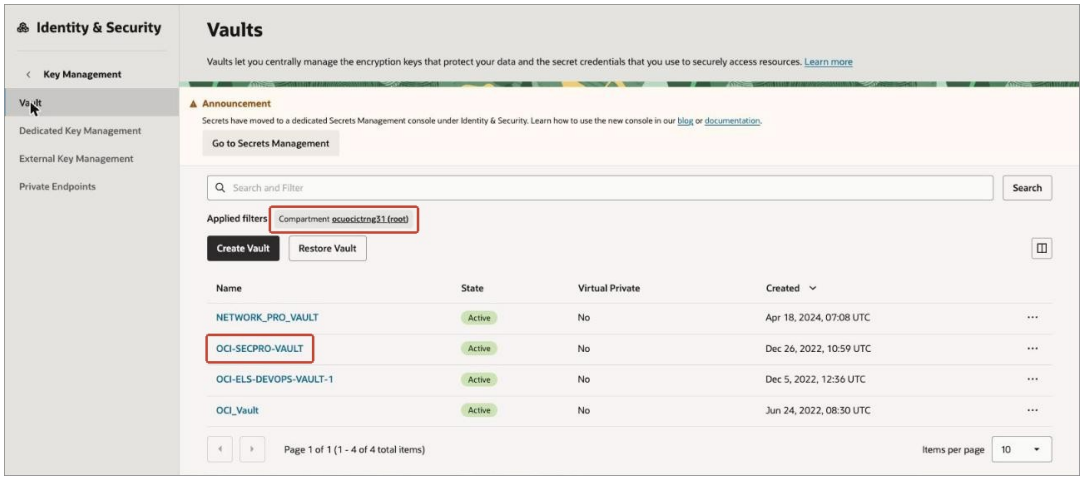



*Figure 2: The shared vault is listed under the root compartment*

3. Click the **Master encryption keys** tab. Next to **Applied filters**, select your assigned compartment, and click **Create Key**.

4. Enter these values, leave the other fields at their defaults, and click **Create Key**:

| Field | Value |
|---|---|
| Create in compartment | Your assigned compartment |
| Protection mode | **HSM** |
| Name | `out1015-userNN-key` |
| Key shape: Algorithm | **AES** |
| Key shape: Length | **256 bits** |

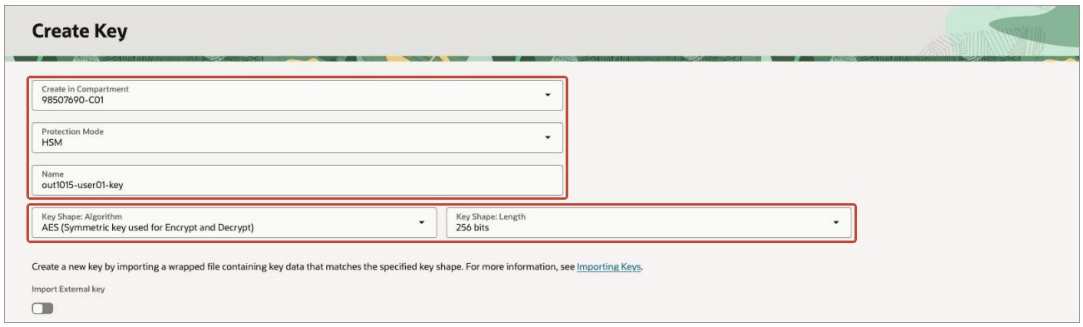



*Figure 3: Create Key: your compartment, HSM protection, and an AES 256-bit key*

5. Wait about a minute, until the key is no longer in the **Creating** state. The key appears in the list.

Copyright © 2026, Oracle and/or its affiliates.

> **Note**
>
> HSM protection means the key never leaves a hardware security module. The vault is shared, but each participant's keys and secrets live in their own compartment.

## Practice 1-2: Store the Password as a Secret

1. From the navigation menu, select **Identity & Security** > **Secret Management**.
2. Next to **Applied filters**, select your assigned compartment and the vault **OCI-SECPRO-VAULT**, and click **Create secret**.
3. Enter these values:

| Field | Value |
|---|---|
| Create in compartment | Your assigned compartment |
| Name | `out1015-userNN-db-password` |
| Description | `Summit Retail order database password (dummy)` |
| Vault compartment | The root compartment |
| Vault | **OCI-SECPRO-VAULT** |
| Encryption key compartment | Your assigned compartment |
| Encryption key | `out1015-userNN-key` |
| Secret generation | **Manual secret generation** |
| Secret type template | **Plain-Text** |
| Secret contents | `SummitDemoOnly-2026!` |
| Cross-region replication | Leave off |

> **Note**
>
> **Vault compartment** starts as your own compartment. Change it to the root compartment first; only then is `OCI-SECPRO-VAULT` listed under **Vault**.

Copyright © 2026, Oracle and/or its affiliates.

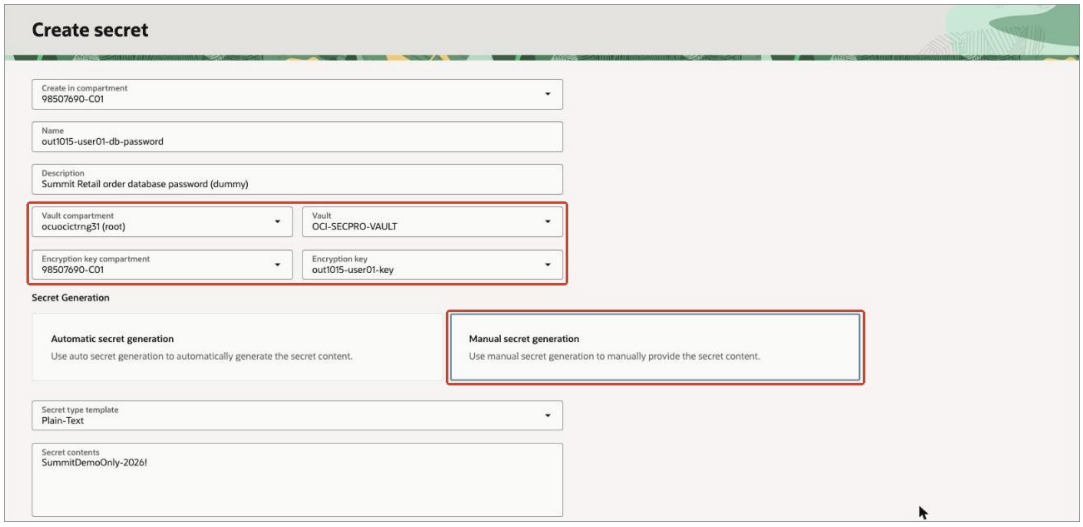



*Figure 4: Create secret: the shared vault in the root compartment, your key, and manual secret generation*

4. Click **Create secret**, and wait until the secret is **Active**.

5. Click `out1015-userNN-db-password`, and copy its **OCID**. It begins with `ocid1.vaultsecret.`. Paste it into your text file; you need it in Practice 1-5.

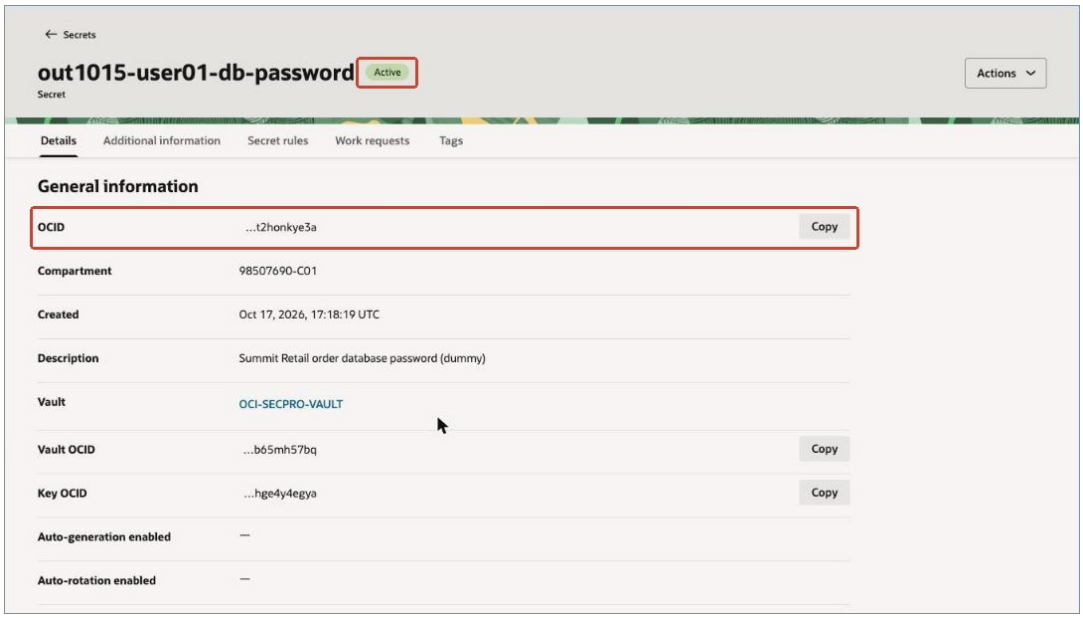



*Figure 5: The secret is Active. Copy the OCID at the top of General information*

> **Caution**
>
> Use only a dummy value in this practice. Never store a real password here.
>
> Copy the Secret OCID, not the vault or key OCID. The application fails with any other kind of OCID.

Copyright © 2026, Oracle and/or its affiliates.

> **Note**
>
> Secret names must be unique in the vault, and OCI keeps a deleted secret's name reserved for several days. If you see **name already exists**, add `-2` to the name, for example `out1015-user07-db-password-2`.

## Practice 1-3: See How the Instance Is Authorized

The application will run without a password and without an API key. OCI gives each instance its own certificate, and the instance signs its requests with it. That identity is the instance principal. Two IAM objects turn it into permission: a dynamic group, whose rule decides which instances belong to it, and a policy that grants the dynamic group access. In this task you only read them.

1. From the navigation menu, select **Identity & Security**, and then click **Policies**. Next to **Applied filters**, select your assigned compartment.

2. Click the policy whose name starts with `POL`, for example `POL98507690-C07`. The lab team created it for your compartment. If an **Introducing IAM Deny** message appears, click **Close**.

3. Click the **Statements** tab. In the search box, enter `SecurityProDynamicGroup`, and click **Search**. Find the two statements about secrets and keys:

```
Allow dynamic-group SecurityProDynamicGroup TO READ secret-family
  IN COMPARTMENT 98507690-CNN
Allow dynamic-group SecurityProDynamicGroup TO USE keys
  in compartment 98507690-CNN
```

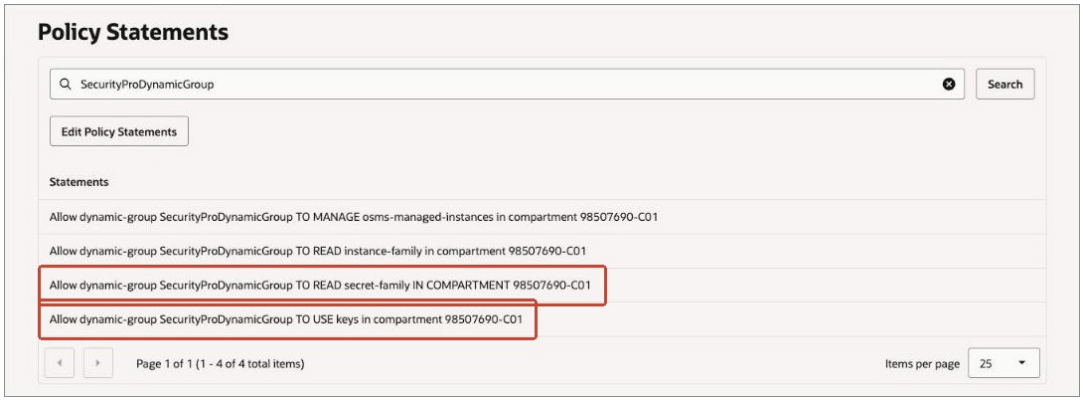



*Figure 6: The policy statements that let the dynamic group read secrets and use keys*

4. Read the first statement in three parts. **Who:** any instance in the dynamic group `SecurityProDynamicGroup`. **What:** read secrets and their contents. **Where:** your compartment.

Copyright © 2026, Oracle and/or its affiliates.

5. The lab team wrote the dynamic group's rule by compartment, so every instance in your compartment belongs to it, including your Tools VM. A rule of that kind looks like this:

```
All {instance.compartment.id = '<your-compartment-ocid>'}
```

> **Note**
>
> Knowing a Secret OCID grants nothing by itself. A request succeeds only when the caller's identity is allowed by a policy.
>
> A rule by compartment includes every instance in that compartment. A tighter rule names one instance OCID or a tag, so that only the order application can read the secret.

## Practice 1-4: Connect to the Tools VM

The application must run on the instance, because only code running there can use the instance's identity. The Tools VM plays the order application's server in this practice.

1. In Cloud Shell, connect to the Tools VM. Replace `<tools_public_ip>` with your value from Practice 0. When SSH asks whether to continue connecting, type `yes`:

```
$ ssh -i ~/out1015-lab/.local/workshop_ssh opc@<tools_public_ip>
```

```
[opc@summit-test ~]$
```

2. Check that the OCI command line interface (CLI) and the Python software development kit (SDK) are installed:

```
$ oci --version
$ python3.9 -c 'import oci; print(oci.__version__)'
```

```
3.x.x
2.x.x
```

> **Note**
>
> From here until the end of Practice 1-6, every command runs on the Tools VM, at the `[opc@summit-test ~]$` prompt. Cloud Shell is only the window you connect from.
>
> If `oci` is not found, the VM is still installing its software. Wait a minute and check again.

Copyright © 2026, Oracle and/or its affiliates.

## Practice 1-5: Retrieve the Password at Runtime

1. On the Tools VM, create a folder and download Oracle's sample application. The address points to a fixed version of the script:

```
$ mkdir -p ~/lab1 && cd ~/lab1
$ SRC=https://raw.githubusercontent.com/ou-developers
$ REV=oci-vaultoperations/f0aac574e5e23e7fd656d7470352d50fd7346870
$ wget -q -O getsecret.py $SRC/$REV/getsecret.py
```

2. Store your Secret OCID in a variable. Replace `<secret-ocid>` with the OCID from Practice 1-2, and keep the single quotes:

```
$ SECRET_OCID='<secret-ocid>'
```

3. Put the OCID into the script, and check the result. The `secret_id` line must show your OCID:

```
$ sed -i "s|ocid1.vaultsecret.oc1.<my_secret_ocid>|$SECRET_OCID|" \
  getsecret.py
$ grep '^secret_id' getsecret.py
```

```
secret_id = "ocid1.vaultsecret.oc1.iad.<your-secret-id>"
```

4. Read the script, and find these lines:

```
$ cat getsecret.py
```

| Line in the script | What it does |
|---|---|
| `secret_id = ...` | Stores the reference to the secret. It is the only secret-related value in the code. |
| `InstancePrincipalsSecurityTokenSigner()` | Signs requests with the instance's own identity. No password or API key is involved. |
| `get_secret_bundle(secret_id)` | Retrieves the current version of the secret. |
| `base64.b64decode(...)` | Decodes the contents. The service returns them Base64-encoded, which is an encoding, not encryption. |

5. Run the order application:

```
$ python3.9 getsecret.py
```

```
SummitDemoOnly-2026!
```

Copyright © 2026, Oracle and/or its affiliates.

> **Expected result**
>
> The script prints the dummy password that you stored in Practice 1-2, although the password appears nowhere in the script or on the VM.

## Practice 1-6 (Optional): Retrieve the Secret with the OCI CLI

1. Retrieve the same secret with the CLI, also as the instance. `--auth instance_principal` tells the CLI to use the instance's identity; the VM has no configuration file or API key:

```
$ oci secrets secret-bundle get --secret-id "$SECRET_OCID" \
  --stage CURRENT --auth instance_principal --region us-ashburn-1 \
  --query 'data."secret-bundle-content".content' --raw-output \
  | base64 --decode; echo
```

```
SummitDemoOnly-2026!
```

2. Leave the Tools VM:

```
$ exit
```

## Checkpoint: What You Proved

The password left the code. The application holds only a reference, and IAM decides at the moment of use whether the instance may read the secret. Answer these questions, then compare your answers:

- Another team receives only your Secret OCID. Can they read the password?
- Another instance is created in your compartment. Can it read your secret?
- An attacker gets a shell on your Tools VM. What can they do with the secret?
- The database password must change. What changes in the application?

Copyright © 2026, Oracle and/or its affiliates.

> **Checkpoint**
>
> **1.** No. The OCID is only a reference. The caller still needs an identity that a policy allows to read the secret.
>
> **2.** Yes, with a rule by compartment: the new instance matches the same rule. Scope the rule more tightly when only one workload needs the secret.
>
> **3.** Code on an authorized instance can use the instance's permissions, so the attacker can read the secret too. Protect access to the host, and give each workload only the secrets it needs.
>
> **4.** Nothing in the code. You add a new secret version, and the application receives it on its next retrieval. The database itself must also change to the new password.

## Learn More

- OCI Vault overview: https://docs.oracle.com/en-us/iaas/Content/KeyManagement/home.htm
- Calling services from an instance (instance principals): https://docs.oracle.com/en-us/iaas/Content/Identity/Tasks/callingservicesfrominstances.htm

Copyright © 2026, Oracle and/or its affiliates.

# Practice 2: Keep the Private Application Private

## Introduction

Summit Retail's order application runs on a VM with no public IP address. Three groups need something from it. Customers use it through the public load balancer. Asha maintains it as an administrator. Every other machine should have no direct path to it at all.

Last night a colleague needed a monitoring script on the Tools VM to reach the application, so they added a quick rule to the private subnet. This morning Asha finds that any machine in the VCN can reach the application directly. In this practice you look at each network control on its own, use the two intended paths, then reproduce the colleague's change and remove it.

**Estimated time:** 30 minutes.

## Objectives

- Identify where a Security List applies and which resources belong to a network security group (NSG).
- Reach the private application as an administrator through an OCI Bastion session.
- Show that customers reach the application through the load balancer only.
- Reproduce the broad rule, see the unintended direct path, and remove the rule without breaking the customer path.

## Prerequisites

- Practice 0 is complete, and you have your `tools_public_ip`, `app_private_ip`, and `order_url`.
- Cloud Shell is open, and you are in the `~/out1015-lab` folder.

## Practice 2-1: Set Up Your Values in Cloud Shell

1. In Cloud Shell, store your values from Practice 0 in variables. Replace each placeholder with your own value; `ORDERS` is the whole `order_url`, starting with `http://`:

```
$ KEY=~/out1015-lab/.local/workshop_ssh
$ TOOLS=<tools_public_ip>
$ APP=<app_private_ip>
$ ORDERS=<order_url>
```

Copyright © 2026, Oracle and/or its affiliates.

> **Note**
>
> Cloud Shell forgets variables when it restarts. If a later command shows an empty address, set the four variables again.

## Practice 2-2: Use the Customer Path

1. Open your `order_url` in a browser tab. The Summit Retail order application page shows **Service is running**.

2. Check the same path from Cloud Shell:

```
$ curl -sS $ORDERS/healthz
```

```
summit-order-app: ok
```

> **Expected result**
>
> Customers reach the application through the public load balancer. The application VM itself has no public IP address.

## Practice 2-3: Find the Rules That Apply to the Private Subnet

A Security List belongs to a subnet. Its rules apply to every VNIC in that subnet.

1. From the navigation menu, select **Networking**, and then click **Virtual cloud networks**. Next to **Applied filters**, select your assigned compartment.

2. Click `out1015-userNN-core-vcn`. Click the **Subnets** tab, and then click `out1015-userNN-core_private`.

3. Click the **Security** tab. In the **Security Lists** section, click `out1015-userNN-core_private-security-list`.

4. Click the **Security rules** tab, and look at both sections:

| Section | What you see | Why |
|---|---|---|
| **Ingress Rules** | No rules | Nothing enters the private subnet because of this list. |
| **Egress Rules** | TCP port 22 to the application's IP address (`/32`) | Lets the Bastion endpoint in this subnet open SSH sessions to the application. |

Copyright © 2026, Oracle and/or its affiliates.

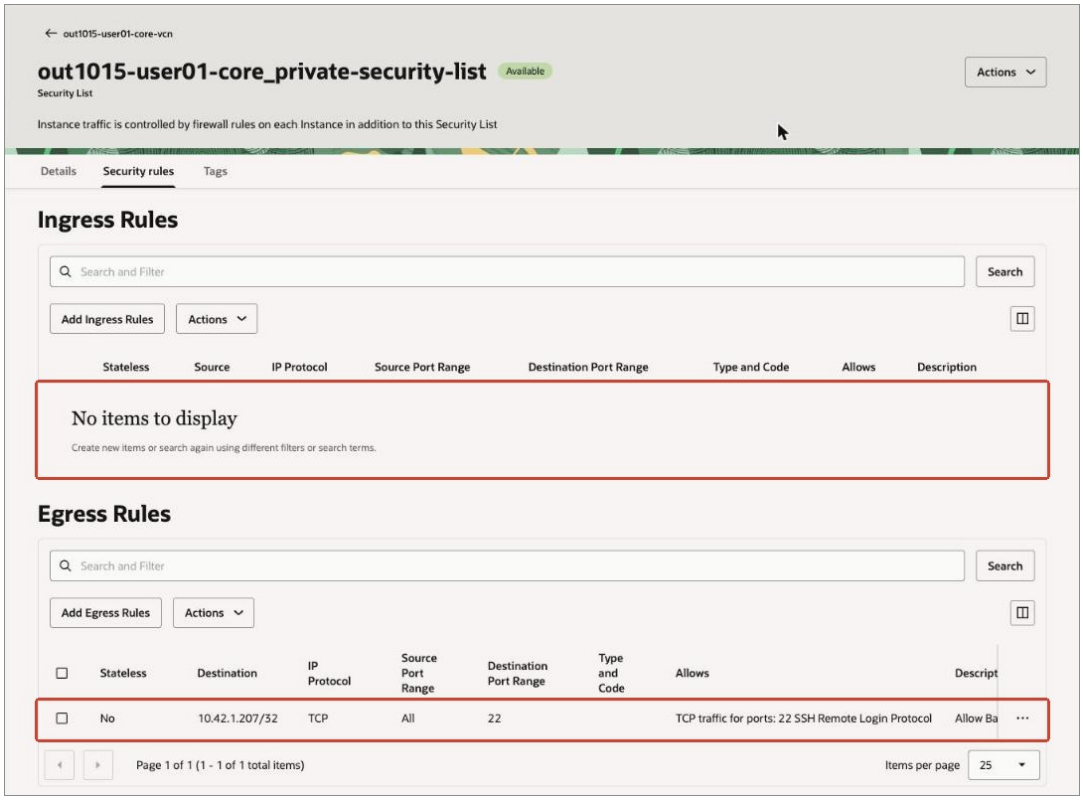



*Figure 7: The private subnet's Security List: no ingress rules, and one egress rule for Bastion SSH*

> **Note**
>
> Keep this page open in a browser tab. You come back to it in Practice 2-6.

## Practice 2-4: Check the Application's Network Security Group

An NSG groups selected resources, wherever they sit in the VCN. A VM belongs to an NSG only if someone adds it.

1. From the navigation menu, select **Compute**, and then click **Instances**. Next to **Applied filters**, select your assigned compartment.

2. Click `out1015-userNN-app`. Click the **Networking** tab, and in the **Primary VNIC** section, find **Network security groups**. The VM belongs to `out1015-userNN-app-nsg`.

3. Click `out1015-userNN-app-nsg`, and then click the **Security rules** tab. Find the two TCP ingress rules:

| Source | Port | Who it lets in |
|---|---|---|
| `out1015-userNN-orders_lb-nsg` (an NSG) | 8080 | The order load balancer, which forwards customer requests |

Copyright © 2026, Oracle and/or its affiliates.

| Source | Port | Who it lets in |
|---|---|---|
| The Bastion endpoint's IP address (`/32`) | 22 | Administrator SSH sessions through Bastion |

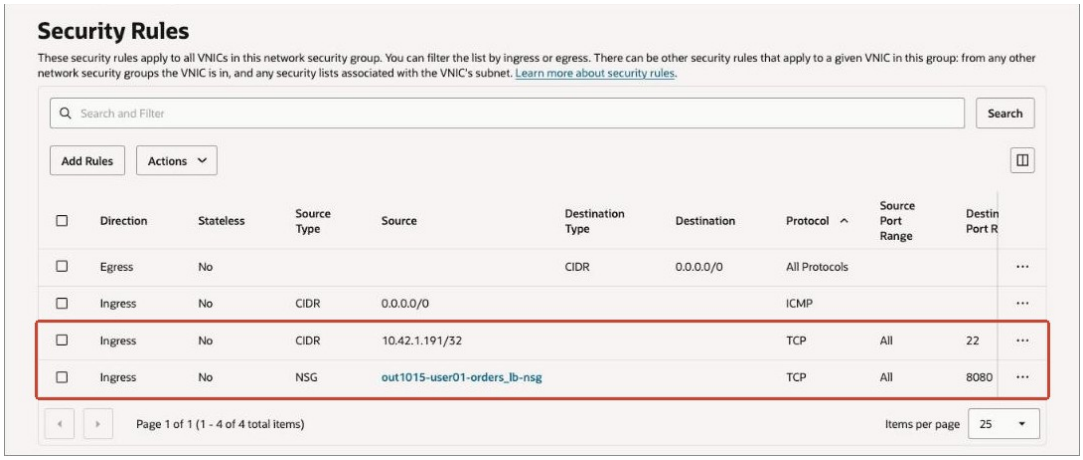



*Figure 8: The application NSG admits only Bastion on port 22 and the load balancer's NSG on port 8080*

> **Expected result**
>
> Only two sources may reach the application: the load balancer on the application port, and Bastion on the SSH port. No rule lets the Tools VM in.

## Practice 2-5: Maintain the Application Through Bastion

Asha needs a shell on the application VM, which has no public IP address. A Bastion session gives her limited-time SSH access without opening the VM to the internet.

1. In Cloud Shell, print your public key, and copy the whole line that starts with `ssh-rsa`:

```
$ cat $KEY.pub
```

```
ssh-rsa AAAAB3NzaC1yc2E... out1015-userNN
```

2. From the navigation menu, select **Identity & Security**, and then click **Bastion**. Next to **Applied filters**, select your assigned compartment.

3. Click `out1015userNNbastion`. Click the **Sessions** tab, and then click **Create session**.

4. Enter these values, and click **Create session**:

| Field | Value |
|---|---|
| Session type | **Managed SSH session** |

Copyright © 2026, Oracle and/or its affiliates.

| Field | Value |
|---|---|
| Session name | `out1015-userNN-admin` |
| Username | `opc` |
| Compute instance compartment | Your assigned compartment |
| Compute instance | `out1015-userNN-app` |
| Add SSH key | **Paste SSH key**, and paste the line that you copied in step 1 |

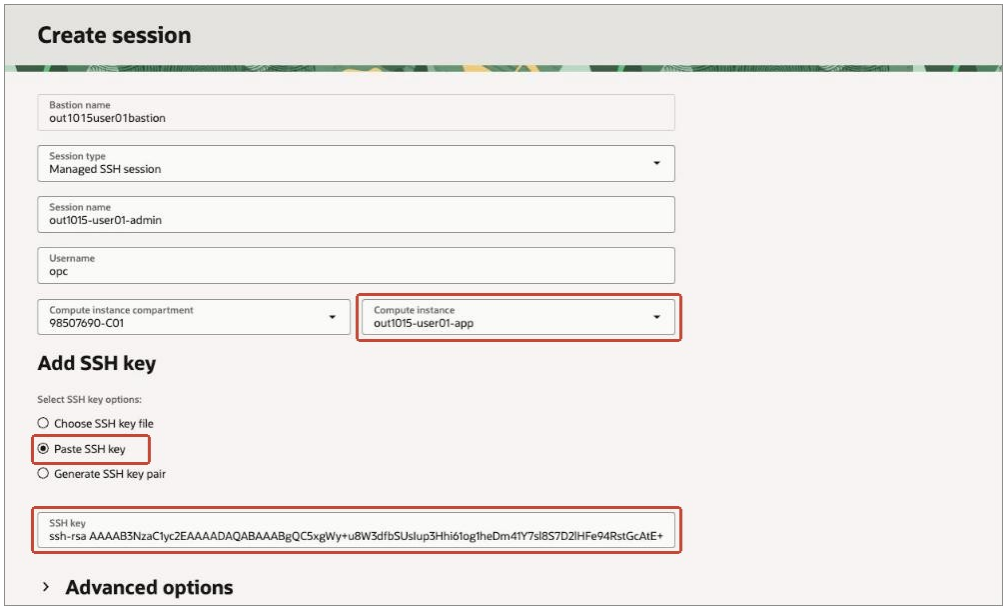



*Figure 9: Create session: a managed SSH session to the order application VM, with your public key*

5. Wait about a minute, until the session is **Active**. The list does not always refresh by itself: if it still shows **Creating**, open the three-dots menu of the session; **View SSH command** is available as soon as the session is Active.

6. Click the three dots next to the session, and click **View SSH command**. Click **Copy**, and then **Close**.

Copyright © 2026, Oracle and/or its affiliates.

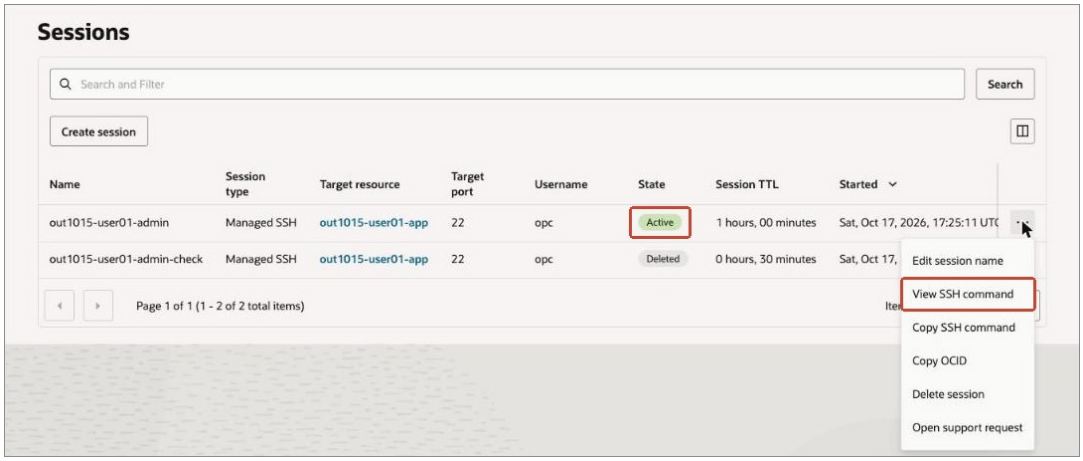



*Figure 10: The session is Active. Open View SSH command from the session's menu*

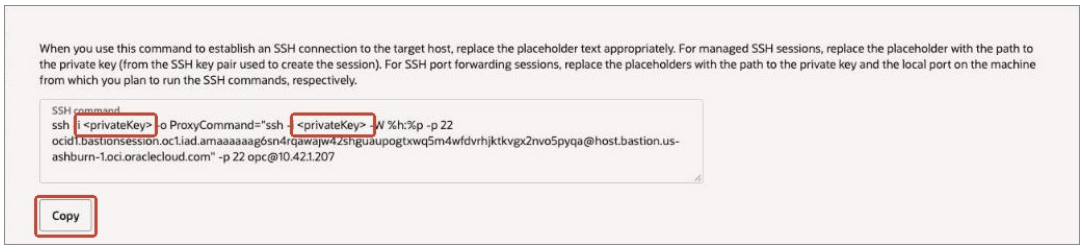



*Figure 11: The SSH command. You replace both privateKey placeholders with $KEY*

7. Paste the command into Cloud Shell, but do not press Enter yet. Replace both `<privateKey>` placeholders with `$KEY`. The command is one long line; here it is split over three lines to fit the page:

```
ssh -i $KEY -o ProxyCommand="ssh -i $KEY -W %h:%p -p 22
ocid1.bastionsession.oc1.iad.<id>@host.bastion.us-ashburn-1.
oci.oraclecloud.com" -p 22 opc@10.42.1.207
```

8. Press Enter. When SSH asks whether to continue connecting, type `yes`. You can be asked twice, once for Bastion and once for the VM.

9. On the application VM, check who and where you are, and whether the application is running:

```
$ hostname; whoami
$ systemctl is-active summit-order-app
$ curl -sS http://127.0.0.1:8080/healthz
$ exit
```

```
summit-app
opc
active
summit-order-app: ok
```

Copyright © 2026, Oracle and/or its affiliates.

> **Expected result**
>
> You reached the private VM as an administrator, through a session that expires on its own after at most one hour.

## Practice 2-6: Reproduce the Incident

Now play the colleague. First confirm that the Tools VM cannot reach the application directly. Then add the colleague's rule and test again.

1. From Cloud Shell, ask the Tools VM to call the application directly. When SSH asks whether to continue connecting, type `yes`:

```
$ ssh -i $KEY opc@$TOOLS \
  "curl -sS --connect-timeout 5 http://$APP:8080/healthz"
```

```
curl: (28) Connection timed out after 5001 milliseconds
```

2. Return to the browser tab with `out1015-userNN-core_private-security-list` from Practice 2-3. On the **Security rules** tab, in the **Ingress Rules** section, click **Add Ingress Rules**.

3. Enter the colleague's rule, and click **Add Ingress Rules**:

| Field | Value |
|---|---|
| Stateless | Do not select |
| Source Type | **CIDR** |
| Source CIDR | `10.42.0.0/16` (the whole VCN) |
| IP Protocol | **TCP** |
| Source Port Range | All |
| Destination Port Range | `8080` |
| Description | `Temporary: monitoring access` |

Copyright © 2026, Oracle and/or its affiliates.

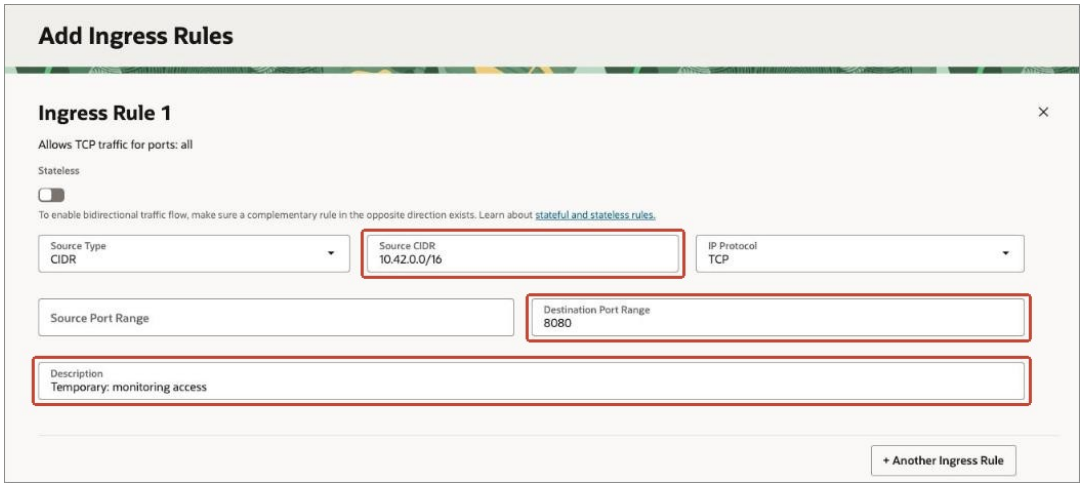



*Figure 12: The colleague's broad rule: the whole VCN on port 8080*

4. Wait a few seconds, and run the same direct test again:

```
$ ssh -i $KEY opc@$TOOLS \
  "curl -sS --connect-timeout 5 http://$APP:8080/healthz"
```

```
summit-order-app: ok
```

> **Expected result**
>
> The Tools VM now reaches the application directly, although the application's NSG did not change. In OCI, the allow rules of the Security List and the NSG add up: traffic that either one allows gets through.

## Practice 2-7: Remove the Broad Rule

1. On the **Security rules** tab of the Security List, find the rule with the description `Temporary: monitoring access`. Click the three dots at the end of its row, and click **Remove**. Confirm when prompted.

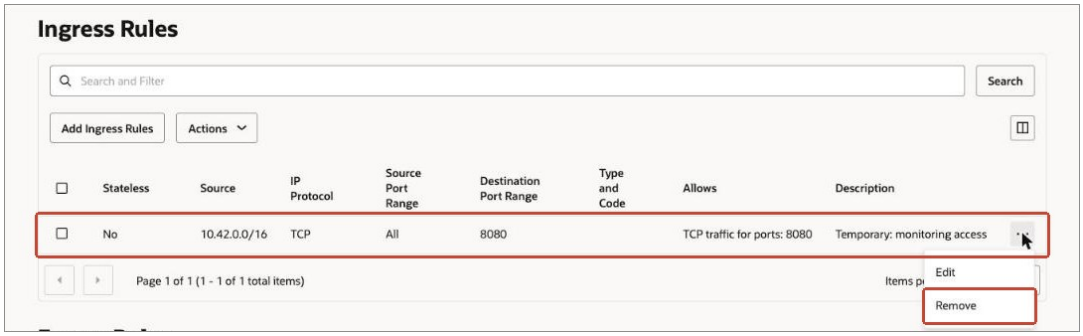



*Figure 13: Remove the temporary rule from the Security List*

Copyright © 2026, Oracle and/or its affiliates.

2. Check that the **Ingress Rules** section is empty again. Do not touch the egress rule on port 22; Bastion needs it.

3. Run the direct test again, and then check the customer path:

```
$ ssh -i $KEY opc@$TOOLS \
  "curl -sS --connect-timeout 5 http://$APP:8080/healthz"
$ curl -sS $ORDERS/healthz
```

```
curl: (28) Connection timed out after 5001 milliseconds
summit-order-app: ok
```

> **Expected result**
>
> The direct path is closed again, and customers still reach the application through the load balancer.

## Practice 2-8: End the Administrator Session

1. In the Bastion's **Sessions** tab, click the three dots next to `out1015-userNN-admin`, and click **Delete session**. Confirm when prompted.

2. (Optional) Run the SSH command from Practice 2-5 again. The session no longer exists, so the connection is refused.

```
Permission denied (publickey).
```

## Checkpoint: What You Proved

| State | Tools VM directly to the app | Customers through the load balancer |
|---|---|---|
| Starting point | Blocked (timeout) | Works |
| With the colleague's rule | **Works (unintended)** | Works |
| After you removed it | Blocked (timeout) | Works |

- Which resources does a rule in the private subnet's Security List affect?
- If you add another VM to the private subnet, does it join `out1015-userNN-app-nsg` automatically?
- The colleague needed monitoring access. What would have been a narrower rule?

Copyright © 2026, Oracle and/or its affiliates.

> **Checkpoint**
>
> **1.** Every VNIC in that subnet, including any VM added later, but nothing in other subnets.
>
> **2.** No. NSG membership is chosen for each VNIC. That is why the NSG is the right place for rules about one application.
>
> **3.** An NSG rule on `out1015-userNN-app-nsg` whose source is the Tools VM's own NSG, or its single IP address (`/32`), on port 8080 only. Better still, monitor through the load balancer's health checks.

## Learn More

- Security rules (Security Lists and NSGs): https://docs.oracle.com/en-us/iaas/Content/Network/Concepts/securityrules.htm
- OCI Bastion overview: https://docs.oracle.com/en-us/iaas/Content/Bastion/Concepts/bastionoverview.htm

Copyright © 2026, Oracle and/or its affiliates.

# Practice 3: Inspect the Requests, Not Only the Port

## Introduction

Summit Retail has opened its product catalogue to customers. The network correctly allows HTTP to the catalogue's load balancer. But an ordinary product search and a request carrying an attack pattern arrive on the same allowed port, and the network cannot tell them apart. Then a script starts asking for the catalogue far more often than any customer would.

Asha's fix is OCI Web Application Firewall (WAF) on the load balancer. In this practice you first watch what one WAF rule detects, then let it block the attack pattern, and finally limit how often one client may request the catalogue. The catalogue is a static page that ignores query strings, so the test pattern never runs; it only shows how WAF recognizes and handles it.

**Estimated time:** 35 minutes.

## Objectives

- Create a WAF policy and attach it to the catalogue's load balancer.
- Detect a cross-site scripting (XSS) pattern in Check mode, and confirm the detection in the WAF log.
- Block the pattern with a 403 response while normal requests still work.
- Limit repeated requests with a 429 response, and watch the catalogue recover.

## Prerequisites

- Practice 0 is complete, and you have your `catalogue_url`.
- Cloud Shell is open.

## Practice 3-1: Meet the Working Shop

1. In Cloud Shell, store your catalogue address and the test pattern, and define a short helper that prints only the HTTP status code. Replace `<catalogue_url>` with your value, which starts with `http://`:

```
$ CAT=<catalogue_url>
$ XSS='q=<script>alert(1)</script>'
$ code() { curl -sS -o /dev/null -w '%{http_code}\n' "$@"; }
```

Copyright © 2026, Oracle and/or its affiliates.

2. Check the catalogue's health:

```
$ curl -sS $CAT/healthz
```

```
summit-retail-web: ok
```

3. In your browser, open two tabs and keep them open for the whole practice:

| Tab | Address |
|---|---|
| Normal customer | `<catalogue_url>/?q=backpack` |
| Test pattern | `<catalogue_url>/?q=%3Cscript%3Ealert%281%29%3C%2Fscript%3E` |

> **Expected result**
>
> Both tabs show the catalogue with three products. Without WAF, the load balancer passes both requests to the web server.

> **Note**
>
> Cloud Shell forgets variables and the helper when it restarts. If a command fails with `code: command not found` or an empty address, run step 1 again.

## Practice 3-2: Create a WAF Policy on the Load Balancer

1. From the navigation menu, select **Identity & Security** > **Web Application Firewall**, and then click **Policies**.

2. Next to **Applied filters**, select your assigned compartment. Click **Create WAF policy**.

3. Enter these values, and click **Create**:

| Field | Value |
|---|---|
| Name | `out1015-userNN-waf-policy` |
| Compartment | Your assigned compartment |

4. Wait until the policy is **Active**. On the policy details page, click the **Firewalls** tab. Click **Actions**, and then **Add Firewalls**.

5. Enter these values, and click **Add Firewalls**:

| Field | Value |
|---|---|
| Firewall name | `out1015-userNN-firewall` |

Copyright © 2026, Oracle and/or its affiliates.

| Field | Value |
|---|---|
| Create in compartment | Your assigned compartment |
| Load balancer compartment | Your assigned compartment |
| Load balancer | `out1015-userNN-waf-lb` |

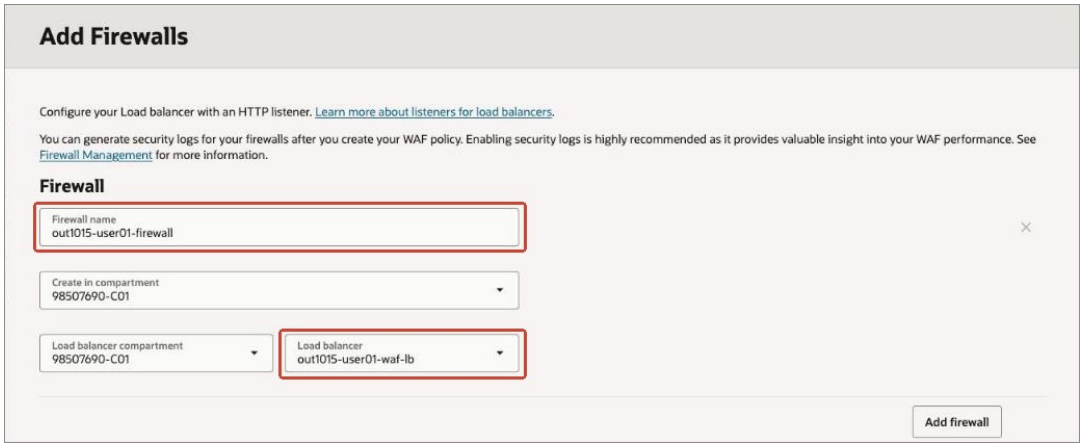



*Figure 14: Add Firewalls: attach the policy to the catalogue load balancer*

6. Wait until the firewall's **Attachment state** is **Active**. It takes a minute or two.

> **Note**
>
> Select the load balancer whose name ends in `waf-lb`. The order load balancer from Practice 2 stays without WAF.

## Practice 3-3: Add an XSS Rule in Check Mode

Check mode records a match but lets the request through. It shows you what a rule would catch before you let it block anything.

1. On the policy details page, click the **Protections** tab. Click **Actions**, and then **Manage request protection rules**. Click **Add request protection rule**.

2. Enter these values:

| Field | Value |
|---|---|
| Name | `inspect-xss` |
| Conditions | Do not add any condition; leave the empty **Path** row as it is |
| Rule action | **Pre-configured Check Action** |
| Body inspection | Leave disabled |

Copyright © 2026, Oracle and/or its affiliates.

3. Under **Protection capabilities**, click **Actions**, and then **Choose protection capabilities**. Next to **Applied filters**, remove the **Tags: Recommended** filter by clicking its **x**; otherwise the search finds nothing. Search for `941110`, select **941110: Cross-Site Scripting (XSS) Attempt: XSS Filters - Category 1**, and click **Choose protection capabilities**.

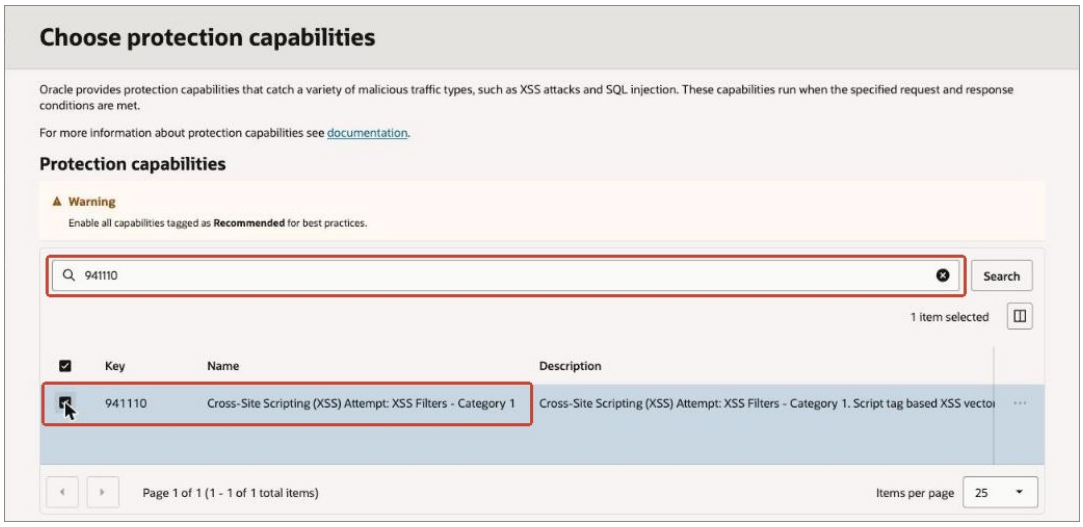



*Figure 15: Choose protection capability 941110*

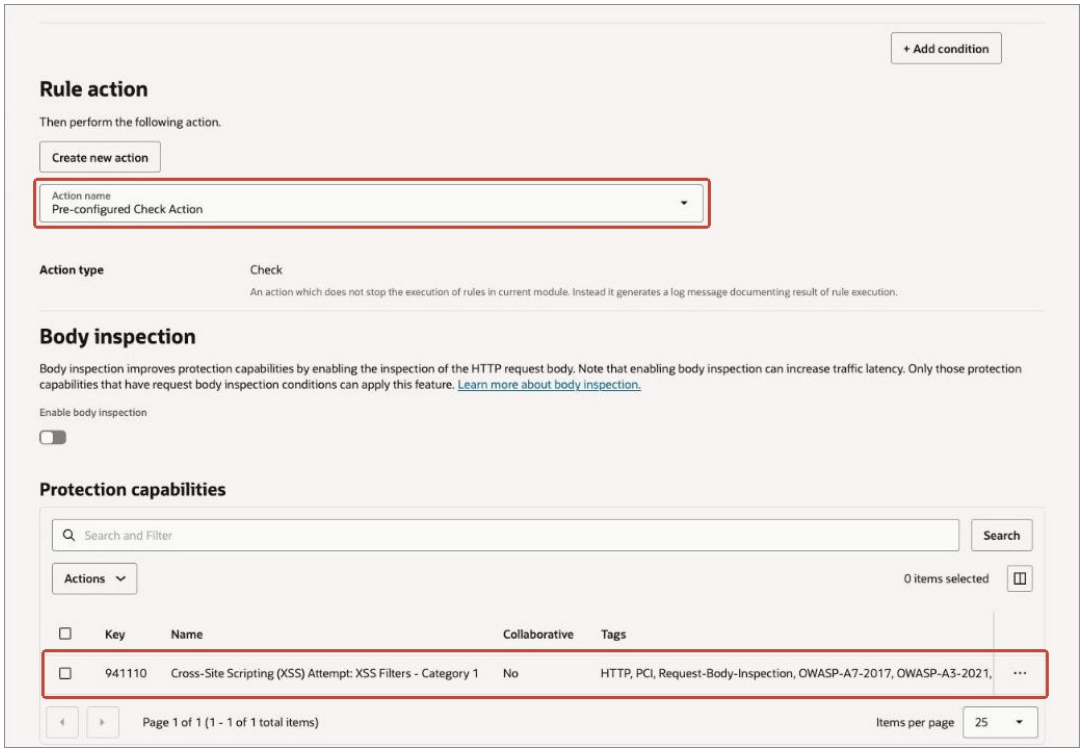



*Figure 16: The rule uses the Pre-configured Check Action and capability 941110*

Copyright © 2026, Oracle and/or its affiliates.

4. Click **Add request protection rule**, and then click **Save changes**. Wait until the policy is **Active** again.

## Practice 3-4: Turn On the WAF Log

1. On the policy details page, click the **Firewalls** tab, and click `out1015-userNN-firewall`.
2. Click the **Monitoring** tab. In the **Logs** section, click the three dots in the **All Logs** row, and click **Enable log**.
3. Enter these values, and click **Enable log**:

| Field | Value |
|---|---|
| Compartment | Your assigned compartment |
| Log group | Create a new group named `out1015-userNN-logs` |
| Log name | Keep the default, `out1015-userNN-firewall_all` |
| Log retention | **1 month** |

4. Wait until the log is **Active**.

## Practice 3-5: Compare Two Requests in Check Mode

1. Reload both browser tabs. Both still show the catalogue.
2. In Cloud Shell, check the exact status codes:

```
$ code "$CAT/?q=backpack"
$ code -G --data-urlencode "$XSS" $CAT/
```

```
200
200
```

3. Open the WAF log: on the firewall's **Monitoring** tab, click `out1015-userNN-firewall_all` in the **Logs** section. The log opens in a new browser tab; click its **Explore log** tab. Log entries can take a few minutes to appear.
4. In the **Search and Filter** box, enter `inspect-xss`, and click **Search**. If nothing is listed, click **Edit** next to **Time range**, select **Past 30 minutes**, and click **Update**.
5. Expand the newest entry, and look at these fields:

| Field | What you see |
|---|---|
| `action` | `allow` |

Copyright © 2026, Oracle and/or its affiliates.

| Field | What you see |
| --- | --- |
| `requestProtection > matchedRules` | `inspect-xss` |
| `requestProtection > matchedIds` | `941110_v002` |
| `response > code` | `200` |

**Expected result**

WAF detected the pattern and let the request through: Check mode records the match and still allows the request. The normal request has no `requestProtection` match.

## Practice 3-6: Block the Pattern

1. On the policy details page, click the **Protections** tab. Click **Actions**, and then **Manage request protection rules**. Click the three dots next to `inspect-xss`, and click **Edit**.

2. Under **Rule action**, click **Create new action**, and enter these values:

| Field | Value |
| --- | --- |
| Name | `block-xss` |
| Type | **Return HTTP response** |
| Response code | **403 Forbidden** |
| Response header | `Content-Type` with value `text/plain; charset=utf-8` |
| Response page body | `Summit Retail WAF: request blocked by XSS rule` |

Copyright © 2026, Oracle and/or its affiliates.

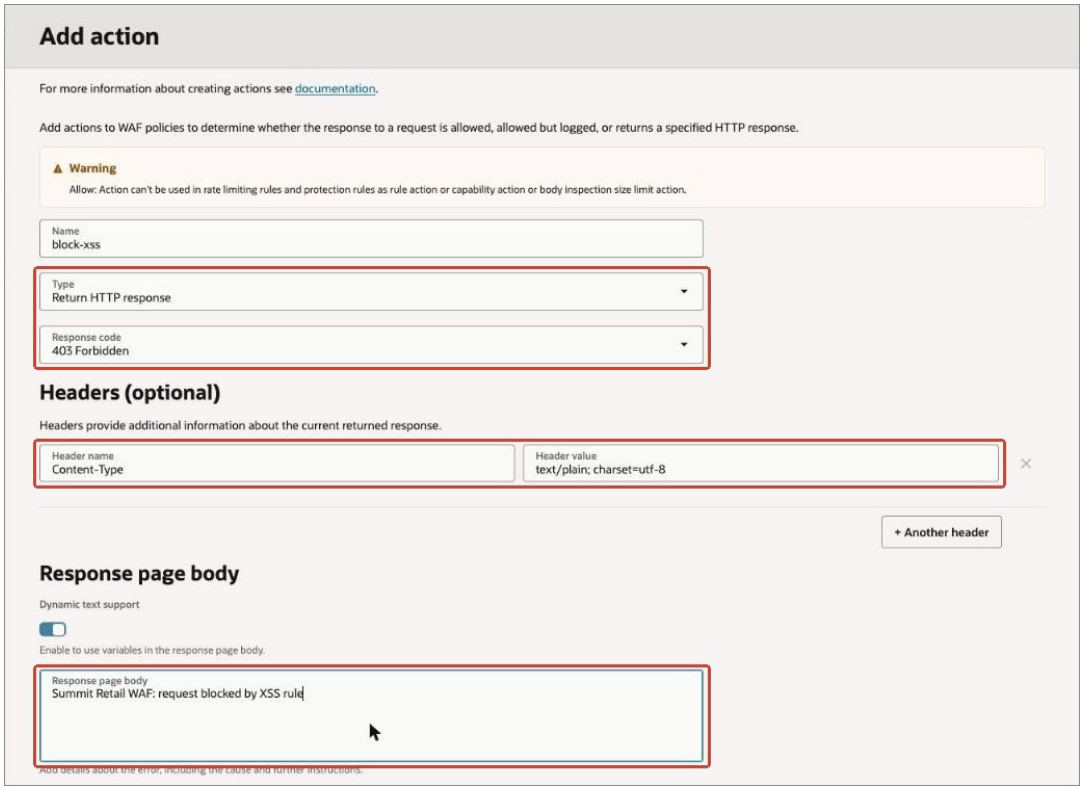



*Figure 17: The block-xss action returns 403 with a plain-text message*

3. Click **Add action**. The new action is now selected under **Action name**. Keep capability 941110 as it is, click **Update**, and then click **Save changes**. Wait until the policy is **Active** again.

4. Test both requests and the health check:

```
$ code "$CAT/?q=backpack"
$ code -G --data-urlencode "$XSS" $CAT/
$ curl -sS -G --data-urlencode "$XSS" $CAT/; echo
$ curl -sS $CAT/healthz
```

```
200
403
Summit Retail WAF: request blocked by XSS rule
summit-retail-web: ok
```

5. Reload the test-pattern tab in your browser with a hard refresh (**Ctrl+Shift+R**, or **Cmd+Shift+R** on a Mac). It now shows the block message, while the normal tab still shows the catalogue.

Copyright © 2026, Oracle and/or its affiliates.

> **Note**
>
> If the pattern still returns `200`, the policy change is still being deployed. Wait a minute and test again.

## Practice 3-7: Limit Repeated Requests

The next request looks ordinary. The problem is how often it comes: a script is asking for the catalogue many times a second.

1. On the policy details page, click the **Rate limiting** tab. Click **Actions**, and then **Manage rate limiting**. Click **Add rate limiting rule**.

2. Enter these values:

| Field | Value |
|---|---|
| Name | `limit-catalogue-requests` |
| Conditions | Condition type **Path**, operator **Is**, value `/` |
| Requests limit | `5` |
| Period in seconds | `10` |
| Action duration in seconds | `30` |

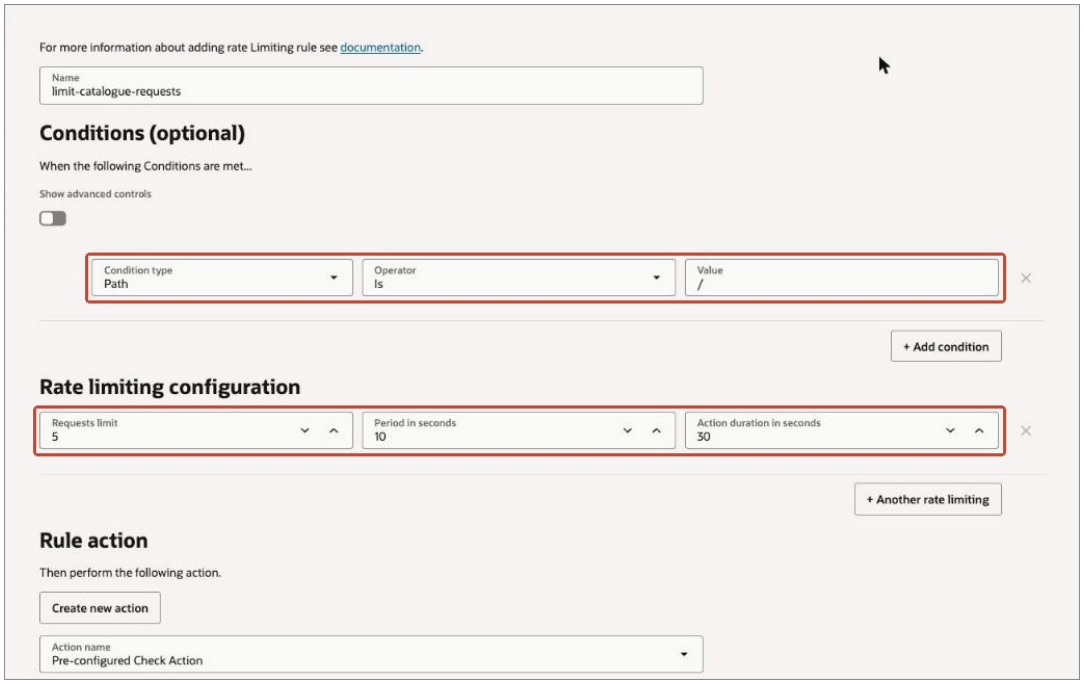



*Figure 18: The rate limiting rule: path /, 5 requests in 10 seconds, then 30 seconds of 429 responses*

Copyright © 2026, Oracle and/or its affiliates.

3. Under **Rule action**, click **Create new action**, and enter these values. Then click **Add action**:

| Field | Value |
|---|---|
| Name | `too-many-requests` |
| Type | **Return HTTP response** |
| Response code | **429 Too Many Requests** |
| Response header | `Content-Type` with value `text/plain; charset=utf-8` |
| Response page body | `Too many catalogue requests. Please try again shortly.` |

4. Click **Add rate limiting rule**, and then click **Save changes**. Wait until the policy is **Active** again.

5. From Cloud Shell, send ten requests in quick succession, then check the health path:

```
$ for i in $(seq 10); do code "$CAT/?q=backpack"; done
$ curl -sS $CAT/healthz
```

```
200
200
200
200
429
200
429
...
summit-retail-web: ok
```

6. Stop sending requests for 40 seconds, then check that the catalogue is back, and that the XSS rule still blocks the pattern:

```
$ code "$CAT/?q=backpack"
$ code -G --data-urlencode "$XSS" $CAT/
```

```
200
403
```




*Figure 19: In Cloud Shell: 429 after a burst, then 200 again after a pause, while the XSS pattern stays at 403*

Copyright © 2026, Oracle and/or its affiliates.

> **Expected result**
>
> After a few ordinary responses, `429` responses appear. The exact sequence varies, because the limit is counted per client and is not exact to the request. `/healthz` stays at `200`, because the rule applies only to the path `/`.

> **Note**
>
> Run the loop from Cloud Shell. Some networks send your requests out through several addresses, which spreads the count and can hide the limit.

## Practice 3-8: Read the Evidence

1. Open the WAF log again, as in Practice 3-5. Clear the search box, click **Search**, and expand the newest entries.

   > **Note**
   >
   > Your load balancer is on the internet, so the log can also hold requests from addresses that are not yours, such as scanners probing common paths. Your own tests come from the Cloud Shell address shown in `clientAddr`.

2. Match each test to its log entry:

| Test | `action` | Response code | Rule |
|---|---|---|---|
| Normal request | `allow` | 200 | None |
| XSS pattern | `block` | 403 | `inspect-xss` (`requestProtection`) |
| Too many requests | `block` | 429 | `limit-catalogue-requests` (`requestRateLimiting`) |

Copyright © 2026, Oracle and/or its affiliates.

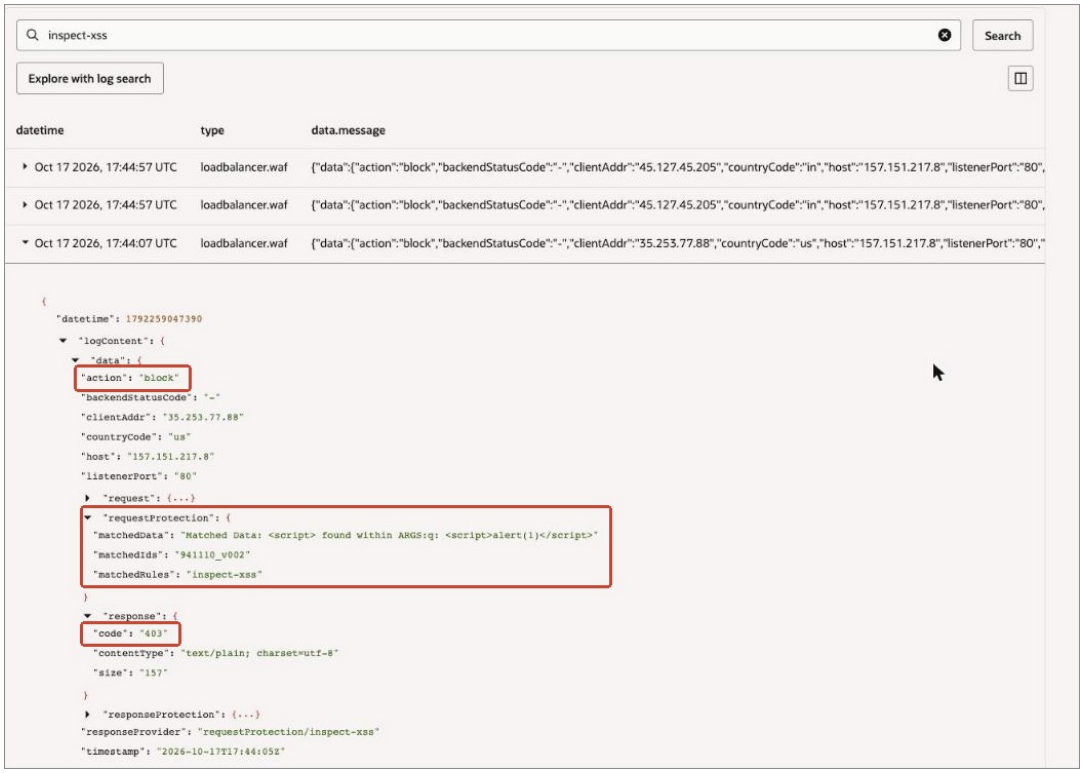



*Figure 20: A blocked request in the WAF log: action block, the matched rule, and response 403*

### Expected result

Each response that you saw in Cloud Shell has a log entry that names the rule and the action. That record is how Asha shows what WAF did, and why.

3. (Optional) Go back to the firewall's **Monitoring** tab. The **Blocked requests** chart counts the 403 and 429 responses over time.

Copyright © 2026, Oracle and/or its affiliates.

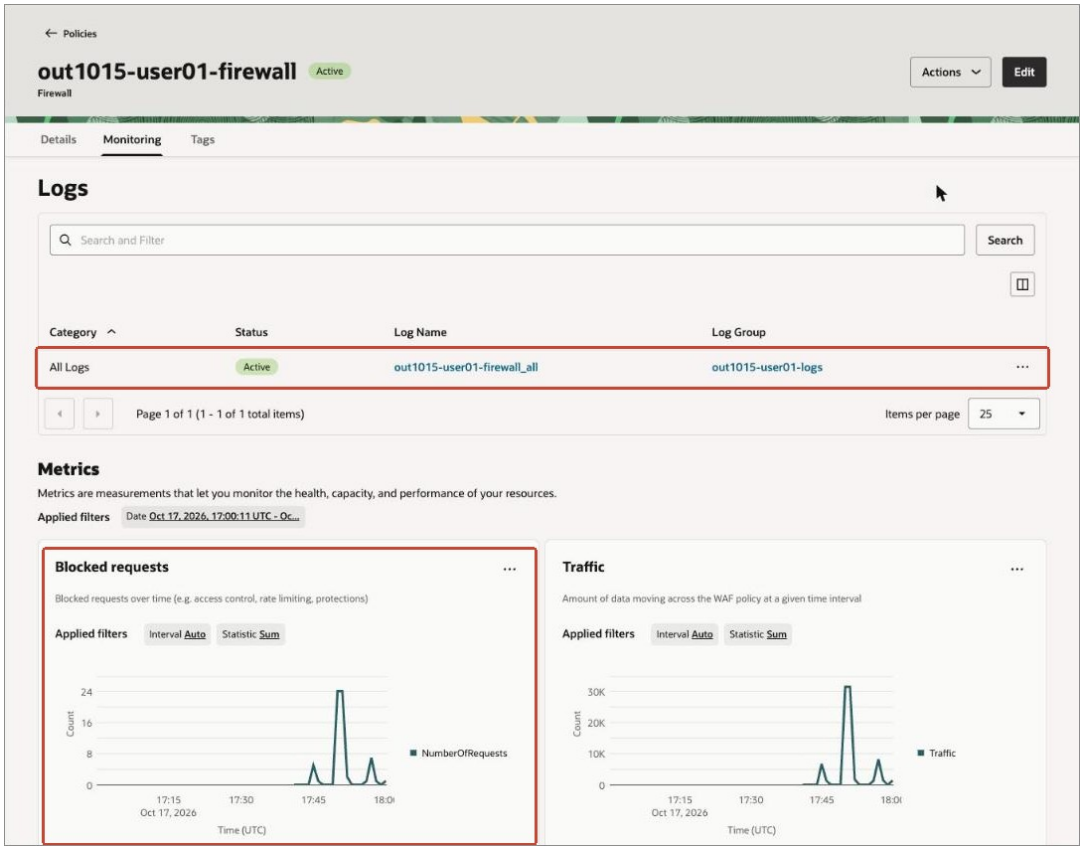



*Figure 21: The firewall's Monitoring tab: the WAF log and the Blocked requests metric*

## Checkpoint: What You Proved

| Request | No WAF | Check mode | Blocking | Rate limit active |
|---|---|---|---|---|
| `?q=backpack` | 200 | 200 | 200 | 200, then 429 |
| XSS pattern | 200 | 200 (logged) | 403 | 403 |
| `/healthz` | 200 | 200 | 200 | 200 |

- Why start in Check mode instead of blocking straight away?
- The XSS rule blocks this pattern. Does that make the application safe from XSS?
- Why does `/healthz` keep working while the catalogue returns 429?

Copyright © 2026, Oracle and/or its affiliates.

> **Checkpoint**
>
> **1.** Check mode shows what a rule matches on real traffic, so you find false positives before customers are blocked.
>
> **2.** No. One rule catches known patterns. The application still needs input handling, patching, and its own security testing.
>
> **3.** The rate rule's condition matches only the path `/`. Health checks and other paths are not counted.

## Learn More

- OCI Web Application Firewall overview: https://docs.oracle.com/en-us/iaas/Content/WAF/Concepts/overview.htm
- Rate limiting rules: https://docs.oracle.com/en-us/iaas/Content/WAF/RateLimiting/create_rate_limiting_rule.htm

Copyright © 2026, Oracle and/or its affiliates.

# Need Help?

Raise your hand, and an instructor will come to you. Before that, check the common problems below.

## Practice 0

- `Expected exactly one active compartment named ...`: check the compartment name in your lab details, and run `python3 lab.py prepare --compartment <name>`.
- `Inputs already exist`: you already ran `prepare` in this folder. Continue with `python3 lab.py plan`.
- `can't open file ... lab.py`: Cloud Shell restarted in your home folder. Run `cd ~/out1015-lab`, and run the command again.
- `Use a fresh participant Cloud Shell ...`: run the package in OCI Cloud Shell, not on your laptop.
- A limit, quota, or capacity error during `apply`: do not retry in another folder. Keep the folder, and tell your instructor.
- `status` reports `not verified` or `Not ready yet`: the servers are still starting. Wait a minute and run `status` again; it can take up to 15 minutes after `apply`.

## Practice 1

- `OCI-SECPRO-VAULT` is not listed: select the root compartment next to **Applied filters**, and check that the region is US East (Ashburn).
- You cannot create the key or secret: check that **Create in compartment** is your assigned compartment, not the root compartment.
- **name already exists** when you create the secret: add `-2` to the name.
- SSH reports `Permission denied (publickey)`: use the key from Practice 0, `~/out1015-lab/.local/workshop_ssh`, and the user `opc`.
- The script still shows `<my_secret_ocid>`: set `SECRET_OCID` again with single quotes, and repeat the `sed` command.
- `NotAuthorizedOrNotFound`: check that you copied the Secret OCID, that the secret is Active, and that the region is US East (Ashburn).

## Practice 2

- An empty address in a command: Cloud Shell restarted and forgot the variables. Set them again as in Practice 2-1.

Copyright © 2026, Oracle and/or its affiliates.

- The Bastion session fails or stays in **Creating**: check that you selected `out1015-userNN-app` and pasted the whole public key line. Delete the session and create a new one.
- SSH through Bastion reports `Permission denied (publickey)`: check that both `<privateKey>` placeholders now read `$KEY`, the user is `opc`, and the session is Active.
- **Create session** fails with `the Bastion plugin must be in the RUNNING state`: the application VM is still starting. In Cloud Shell, run `cd ~/out1015-lab && python3 lab.py status`, and create the session once it reports `Environment ready.`
- SSH reports `REMOTE HOST IDENTIFICATION HAS CHANGED`: your Cloud Shell remembers an older VM with the same address, for example after you rebuilt Practice 0. Run `ssh-keygen -R <address>` with the address from the message, and connect again.
- SSH through Bastion times out: check that the session is Active and the egress rule on port 22 is still in the private subnet's Security List.
- The direct test works before you add the rule: look for an extra ingress rule in the private subnet's Security List, and tell your instructor.
- The direct test still works after you removed the rule: wait a few seconds, and check that you removed the rule with the description `Temporary: monitoring access`.

## Practice 3

- `code: command not found`: Cloud Shell restarted. Run Practice 3-1, step 1 again.
- `502 Bad Gateway`: the web server is not ready or not healthy. Check `curl -sS $CAT/healthz`, wait a minute, and tell your instructor if it does not recover. A 502 is not a WAF block.
- The XSS pattern still returns `200` after Practice 3-6: check that `inspect-xss` uses the action `block-xss`, that the policy is Active, and that the firewall is attached to `out1015-userNN-waf-lb`. Then send a fresh request.
- No `429` in Practice 3-7: check that the rule's condition is the path `/`, wait until the policy is Active, and run the loop again from Cloud Shell.
- The log is empty: check that the log is enabled on the firewall, and wait a few minutes. Log entries arrive after the HTTP response.

Copyright © 2026, Oracle and/or its affiliates.

# Clean Up

Clean up when your instructor tells you to, normally at the end of the day. The lab resources incur charges while they exist.

## Check for Leftover Lab Changes

1. In your Bastion's **Sessions** tab, delete any session that is still **Active**.
2. In `out1015-userNN-core_private-security-list`, check that the **Ingress Rules** section is empty. Remove the rule `Temporary: monitoring access` if it is still there.

## Remove Your WAF Policy and Log

1. From the navigation menu, select **Identity & Security** > **Web Application Firewall**, and then click **Policies**. Next to **Applied filters**, select your assigned compartment.
2. Click `out1015-userNN-waf-policy`. Click the **Firewalls** tab, select `out1015-userNN-firewall`, and click **Actions** > **Delete**. Confirm, and wait until the firewall is deleted.
3. At the top of the policy page, click **Actions** > **Delete**, and confirm.
4. From the navigation menu, select **Observability & Management** > **Logging**, and then click **Log Groups**. Click `out1015-userNN-logs`, delete the log `out1015-userNN-firewall_all`, and then delete the log group.

## Delete Your Secret and Key

1. From the navigation menu, select **Identity & Security** > **Secret Management**. Next to **Applied filters**, select your assigned compartment and **OCI-SECPRO-VAULT**.
2. Click `out1015-userNN-db-password`, and then click **Actions** > **Delete Secret**.
3. Select the confirmation box, type the secret name, keep or choose the earliest deletion date, and click **Delete Secret**.
4. From the navigation menu, select **Identity & Security** > **Key Management**, click **Vault**, and then click **OCI-SECPRO-VAULT**. On the **Master encryption keys** tab, select your assigned compartment next to **Applied filters**.
5. Click `out1015-userNN-key`, and then click **Actions** > **Delete Key**.

Copyright © 2026, Oracle and/or its affiliates.

6. Select the confirmation box, type the key name, keep or choose the earliest deletion date (at least 7 days ahead), and click **Delete Key**.

> **Note**
>
> OCI deletes keys and secrets only after a waiting period, which protects against mistakes. Their state shows **Pending Deletion** until then.

## Remove the Practice 0 Environment

1. In Cloud Shell, go to your Practice 0 folder and review what would be deleted:

```
$ cd ~/out1015-lab
$ python3 lab.py destroy-plan
```

```
Reviewed plan scope: {"create": 0, "update": 0, "delete": 48}
```

2. Delete the resources. When asked, type your prefix, for example `out1015-user07`:

```
$ python3 lab.py destroy
```

```
No Terraform-managed resources remain in this lab state.
```

Copyright © 2026, Oracle and/or its affiliates.

# Acknowledgements


- **Author:** Saurabh Patil, Principal OCI Instructor, Oracle University
- **Workshop:** OUT1015 Zero Trust Security with OCI, Oracle AI World 2026
- **Last updated:** October 2026


Copyright © 2026, Oracle and/or its affiliates.